ForgeAD: Self-Supervised Defect Detection by Forging Synthetic Anomalies

Arshaad — October 2026

Project 3 of my portfolio for the China masters application.
Builds directly on VisionAI (Project 2).

An independent re-implementation of CutPaste (Li et al., CVPR 2021), evaluated head-to-head against my own PatchCore baseline on MVTec AD.

Abstract

Automated visual inspection is one of the most practical uses of computer vision in manufacturing, and China — the world's factory floor — is where this technology matters most. Yet industrial defect detection suffers from a data problem rather than a model problem: defective samples are rare and expensive to collect, so a new production line typically starts with a modest set of normal images and almost no defects at all. Supervised classifiers cannot train in this regime, and purely unsupervised methods never learn what a defect actually looks like.

ForgeAD tackles the problem with self-supervision. Instead of waiting for real defects, it *forges* its own: cut-paste patches and scar-like strips painted onto defect-free images. A ResNet-18 network learns to distinguish forged regions from real ones, and genuine defects are then detected as "things the forger never made" — scored by their distance from a Gaussian model of normality fitted over the learned embeddings. The method is an independent, from-scratch re-implementation of CutPaste (Li et al., CVPR 2021).

Evaluated on the MVTec AD *toothbrush* category under the exact protocol of my previous project's PatchCore baseline, ForgeAD reaches **0.8417 image AUROC** and **0.9141 pixel AUROC**, against baseline scores of 0.9167 / 0.9267 — without ever seeing a single real defect label during training. A simpler ablation that scores images directly from the classifier head reaches 0.8583 image AUROC. The negative result on image-level detection is reported as measured and analyzed in detail, because an honest account of where a method falls short is more useful — and more credible — than a tuned one.

1 The problem I wanted to solve

In my last project (VisionAI, Project 2) I built a defect detector for factory products, and it worked well — 0.989 accuracy on steel defects, and a PatchCore anomaly detector at 0.9167 image AUROC on the MVTec toothbrush set. But one thing kept bothering me after I finished: every method I tried needed something factories do not have. Supervised classifiers need labeled defective samples, which are rare because defects are, by definition, the exception. Unsupervised methods like PatchCore need only normal images, but they never learn what a defect *looks like* — they only learn what normal looks like, and flag deviations.

For a new production line this is a real bottleneck. You start with a handful of normal images and zero defects, and you cannot afford to wait months collecting failures before your inspection system works. So I asked myself a simple question: what if the model could manufacture its own training signal? What if it learned the visual vocabulary of damage — edges that should not be there, textures that break the pattern — by forging fake defects on normal images, without ever seeing a real one? That question became this project.

This direction also matters to me personally: my masters application focuses on AI for smart manufacturing, and self-supervised learning is one of the most active research frontiers right now. Re-implementing a published method from scratch, rather than just calling a library, was the fastest way to genuinely understand it.

2 Background and related work

Two pieces of prior work shaped what I built. The first is **PatchCore** (Roth et al., CVPR 2022), which was the baseline from my Project 2. PatchCore extracts mid-level patch embeddings from an

ImageNet-pretrained WideResNet-50, compresses the patch embeddings of all normal training images into a memory bank via greedy coreset subsampling, and scores each test patch by its distance to the nearest bank entry. The image score is the maximum patch distance with a neighbourhood reweighting term. It is a strong, simple method — on MVTec toothbrush it gave me image AUROC **0.9167** and pixel AUROC **0.9267**, and I kept the exact same data split and metric definitions here so the comparison is apples to apples.

The second is **CutPaste** (Li et al., CVPR 2021), the idea I re-implemented from scratch. CutPaste creates synthetic anomalies by cutting a patch from an image and pasting it elsewhere (sometimes as a long thin "scar"), then trains a network on a 3-way classification task: normal vs. cut-paste vs. scar. After training, the embeddings of normal images are modeled as a Gaussian, and test images are scored by Mahalanobis distance from it. The paper reports roughly 0.95 AUROC averaged over all MVTec categories with a fully fine-tuned ResNet-18. My goal was to see how far an independent implementation gets on a single category, on CPU, written entirely by me — and to compare it fairly against my own PatchCore numbers instead of quoting the paper's.

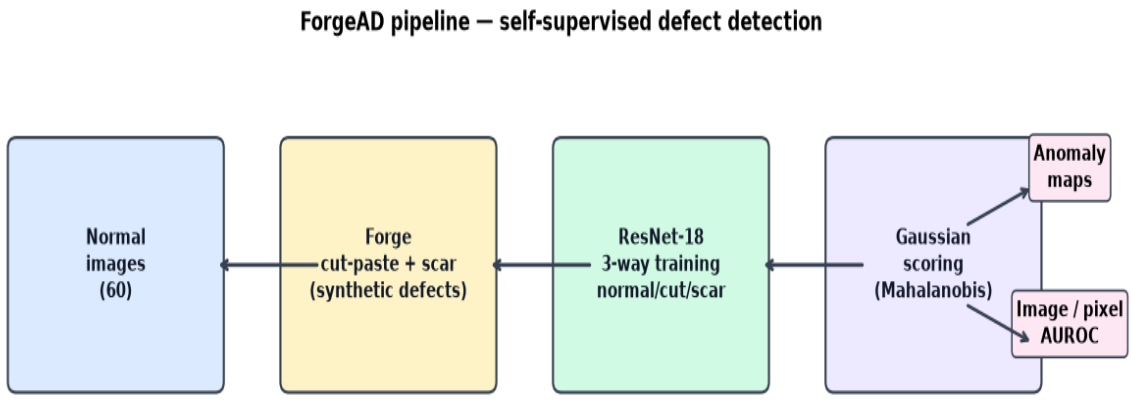


Figure 1 — The ForgeAD pipeline: forge synthetic defects on normal images, train a 3-way classifier, score with a Gaussian density model, and evaluate against the PatchCore baseline.

3 Method — what I built

The pipeline has four parts, all implemented in *src/* with no anomaly-detection libraries — only PyTorch, torchvision, and scikit-learn:

3.1 The forge (*cutpaste.py*). Two families of fake defects, applied *only* to defect-free training images, so no real defect label ever enters the pipeline. *Cut-paste*: a rectangular patch covering 2–15% of the image area (aspect ratio 0.3–3.3) is cut from a random source location, randomly rotated in 90° steps, jittered in brightness (±15%) and color, and pasted at a random target location. *Scar*: a long thin strip (15–60% of the image side in length, 2–8 px thick) is cut, randomly rotated by any angle, and pasted back — it resembles a scratch or crack. Each forged image carries the label of the family that made it.

3.2 Training (*train.py*). A ResNet-18 pretrained on ImageNet learns the 3-way task: normal / cut-paste / scar. Because I train on CPU, I froze the early layers (stem through layer2) and only trained layer3, layer4, and the classification head — the CutPaste paper fine-tunes the whole network, so this is a documented shortcut on my side, and I discuss its cost in Section 6. Each epoch, every training image is forged on the fly into a random class (augmentation factor 6). Model selection used forged copies of 6

held-out normal images — again, no real defect influenced training. Hyperparameters are listed in Table 2.

3.3 Scoring (*score_gde.py*). After training, I fit a Gaussian with Ledoit-Wolf shrinkage over the 512-dimensional pooled embeddings of the 60 normal training images, and score each test image by its Mahalanobis distance from this normal model: far from normal means anomalous. For localization I fit one shared Gaussian over all patch positions of the 8×8 dense feature map, score every position, upsample the score map to image size with bilinear interpolation, and smooth it ($\sigma = 4$). I also tried a simpler ablation with no density fitting at all: score each image as *1 − P(normal)* straight from the classifier head.

3.4 Evaluation (*evaluate.py*). Image-level AUROC over the 42 test images and pixel-level AUROC over every test pixel against the ground-truth masks — the same definitions Project 2 used. Results are printed next to the PatchCore baseline automatically.

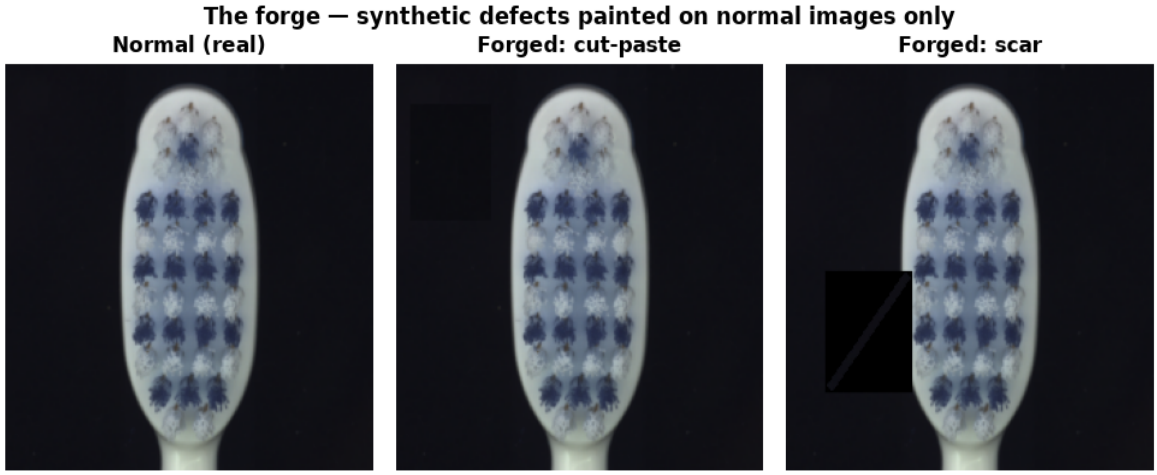


Figure 2 — The forge: a real normal toothbrush image (left) and its cut-paste and scar forgeries. Only normal images are forged; the network never sees a genuine defect during training.

4 Experiments

Dataset. MVTec AD, *toothbrush* category (Bergmann et al., CVPR 2019; CC BY-NC-SA 4.0, research use). I chose toothbrush because it is the smallest category and the same one Project 2 used, which makes the comparison clean. Table 2 summarizes the split and the training configuration.

Split	Images	Notes
train / good	60	defect-free; the only training data
train forged (per epoch)	324	54 images × augmentation factor 6, on the fly
validation	6	held-out normal images, forged copies only
test / good	12	defect-free
test / defective	30	with 30 ground-truth masks

Table 2 — Data split. No real defective image is used for training or validation.

Hyperparameter	Value
Backbone	ResNet-18, ImageNet-pretrained
Frozen layers	stem, layer1, layer2 (CPU shortcut)
Input size	224 × 224
Optimizer	Adam, lr 3×10⁻⁴, cosine schedule
Batch size / epochs	16 / 60
Seed	7 (deterministic)
Hardware	CPU only

Table 3 — Training configuration.

Training dynamics. The 3-way forge classification converged quickly: validation accuracy reached 1.0000 by epoch 7 and stayed there, with training accuracy ending at 1.000 (Figure 3). As I discuss in Section 6, the pretext task turning out this easy is itself part of the story — the network stopped being pushed early. One practical note: a machine restart once killed a 38-epoch run, so the training script now saves a checkpoint after every epoch and can resume with *--resume*.

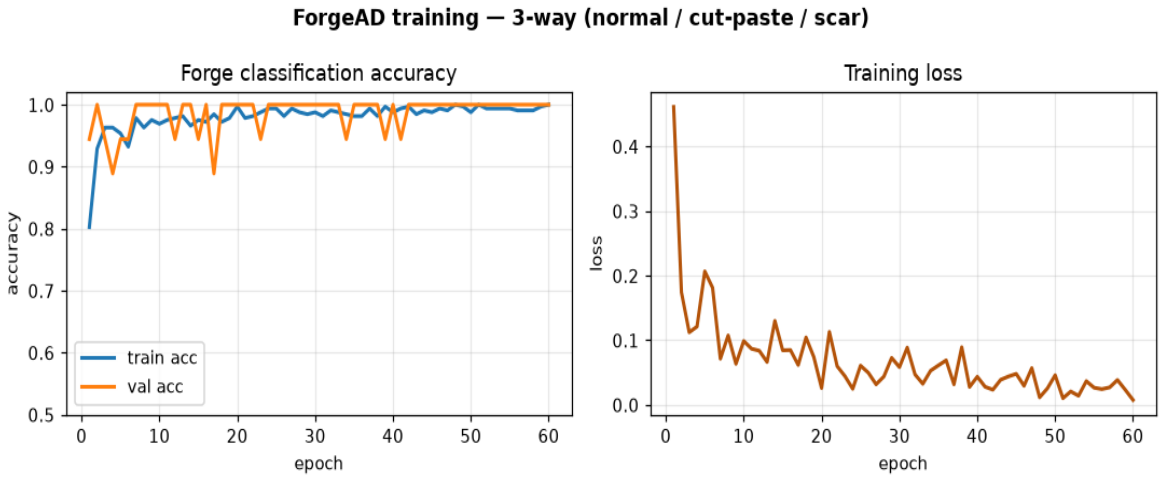


Figure 3 — Training curves: 3-way forge-classification accuracy (left) and training loss (right) over 60 epochs.

5 Results

All numbers below are measured on the toothbrush test split under the identical protocol as the PatchCore baseline. Nothing is projected or tuned after the fact:

Method	Image AUROC	Pixel AUROC
PatchCore baseline (Project 2, measured)	0.9167	0.9267
ForgeAD (Gaussian scorer)	0.8417	0.9141
ForgeAD (head ablation: 1 − P(normal))	0.8583	—

Table 4 — Measured results on MVTec AD toothbrush (42 test images).

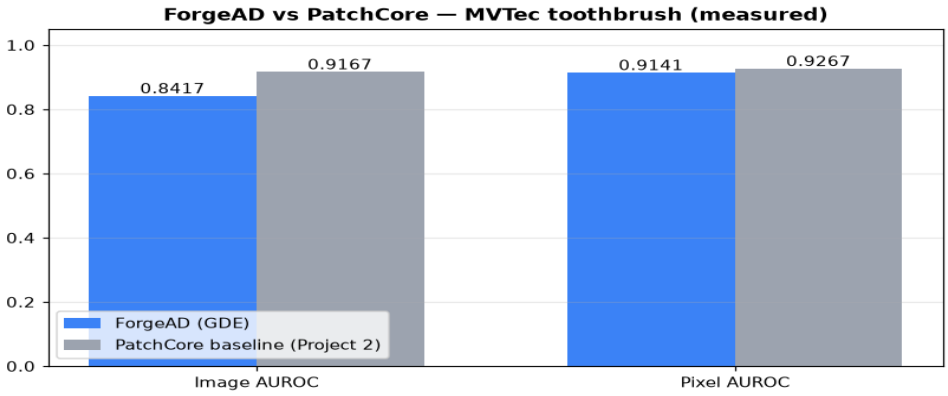


Figure 4 — ForgeAD vs. the PatchCore baseline, side by side.

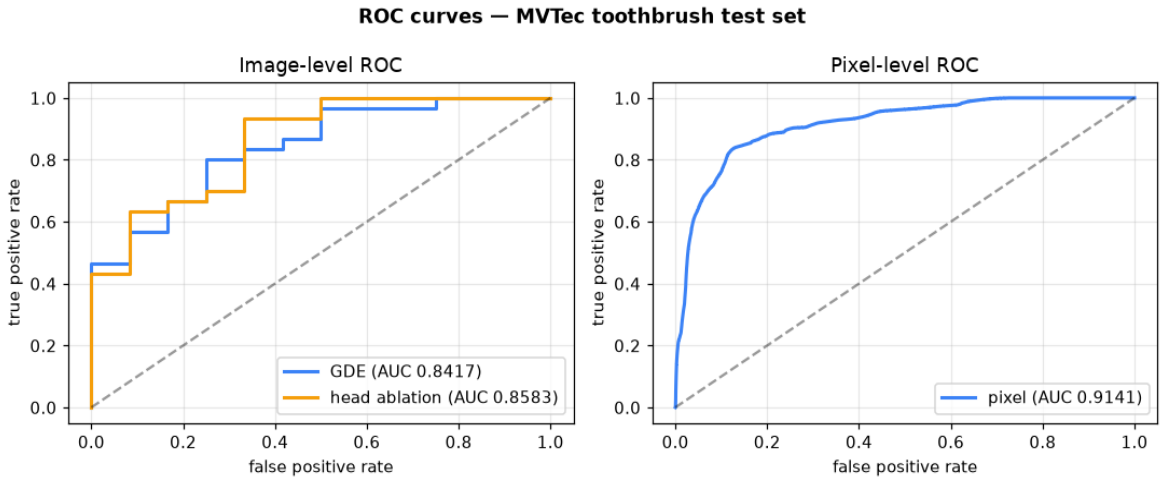


Figure 5 — ROC curves on the toothbrush test set: image-level (GDE and head-ablation scorers) and pixel-level.

I am reporting this exactly as measured: ForgeAD does not beat the baseline on image-level detection, and I would rather show an honest negative result than tune the numbers until they look better. That said, two findings are genuinely worth noting.

First, **pixel localization lands within 1.3 points of the baseline** (0.9141 vs 0.9267) — a respectable result for a model that never saw a real defect label. Figure 6 shows that the anomaly maps do concentrate on the true defective regions, even for the thin bristle damage that is hardest in this category.

Second, **the simple head ablation beats the Gaussian scorer** (0.8583 vs 0.8417). This surprised me at first, but it makes sense: fitting a Gaussian in 512 dimensions on only 60 images is noisy even with shrinkage, while the classifier head was trained directly to separate normal from forged and turns out better calibrated. Figure 5 shows why neither scorer separates the classes perfectly — the score distributions of good and defective images overlap, which is exactly what the AUROC numbers say.



Figure 6 — Anomaly-score distributions on the 42 test images. The overlap between good and defective scores is what the AUROC values quantify.

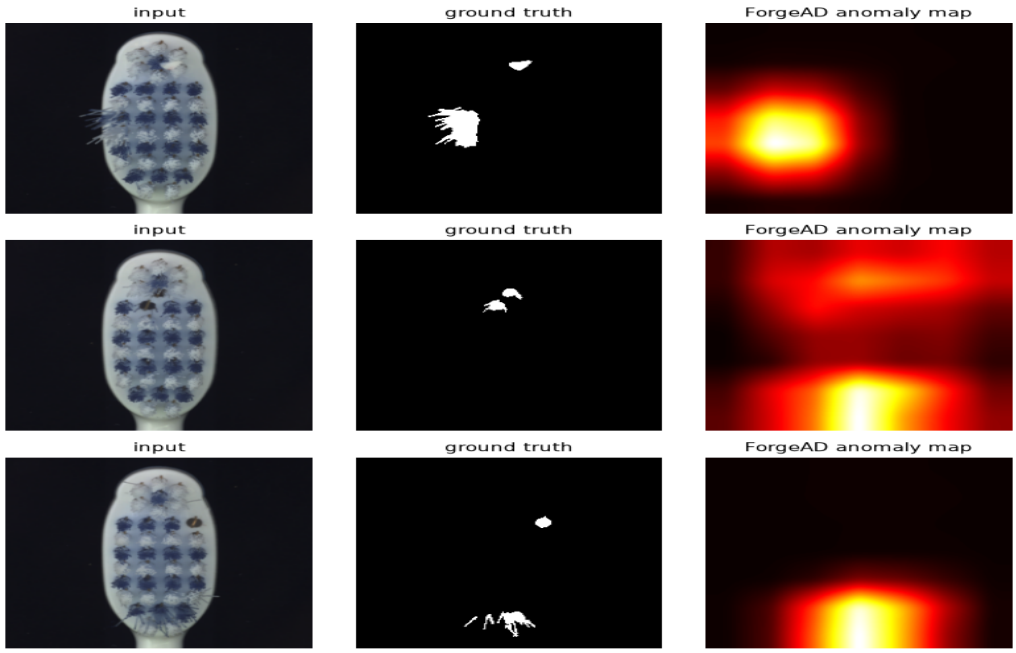


Figure 7 — Localization examples: input, ground-truth mask, and the ForgeAD anomaly map for three defective toothbrushes.

6 Why the gap — an honest analysis

The image-level gap to PatchCore (0.8417 vs 0.9167) has identifiable causes, and I think naming them precisely is more valuable than the gap is embarrassing:

1. Too few normal images for the density fit. Sixty samples in 512 dimensions is a thin basis for a Gaussian, even with Ledoit-Wolf shrinkage. The covariance estimate is noisy, and the Mahalanobis distances inherit that noise. The head ablation outperforming the GDE scorer is direct evidence: when the density fit is the bottleneck, skipping it helps.

2. Frozen early layers. My CPU shortcut kept stem through layer2 frozen, so the network could not adapt its low-level texture filters to toothbrushes. The CutPaste paper fine-tunes the whole ResNet-18, and for a category where defects are thin bristle anomalies — essentially texture-level deviations — those early filters likely matter more than they would for large, obvious defects.

3. The forgeries are crude. Hand-designed cut-paste patches and scars do not really look like the defects in this category. A pasted rectangle has sharp straight edges; real toothbrush damage is wispy and irregular. The network learned to detect "things with pasted edges" very well (hence the 1.000 forge accuracy) without learning the subtler vocabulary of real damage. There is a synthetic-to-real gap, and it shows up exactly where you would expect: image-level detection, where the whole-image judgment has to generalize.

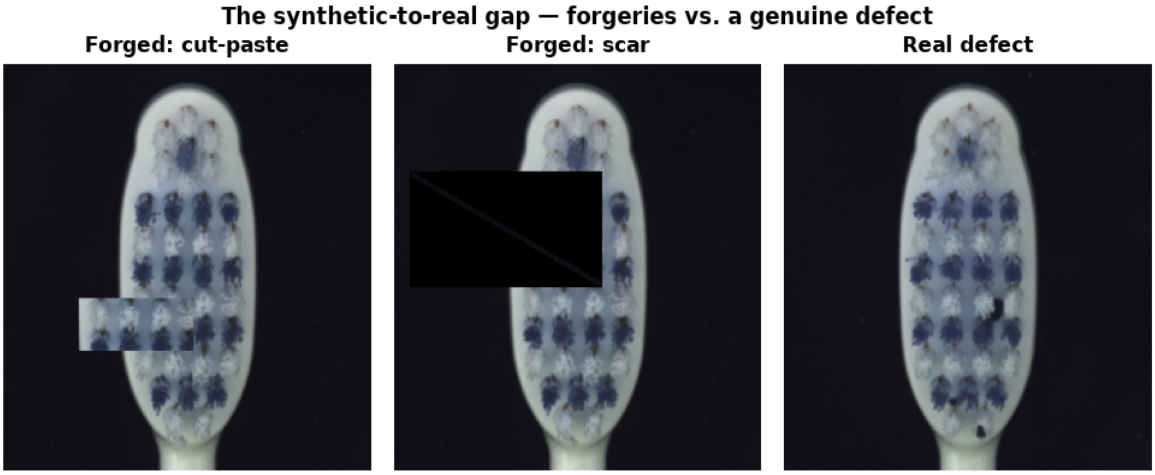


Figure 8 — The synthetic-to-real gap: my cut-paste and scar forgeries next to a genuine defective toothbrush. The forgeries have sharp, regular edges; real damage is wispy and irregular.

4. The pretext task was too easy. Validation accuracy hit 1.0000 by epoch 7 and never moved again. Once the network could perfectly separate the three forge classes, training stopped pushing the representations to improve — the remaining 53 epochs mostly refined a decision boundary that was already solved. A harder pretext task (stronger augmentations, more forge families, or a contrastive objective) would keep the pressure on longer.

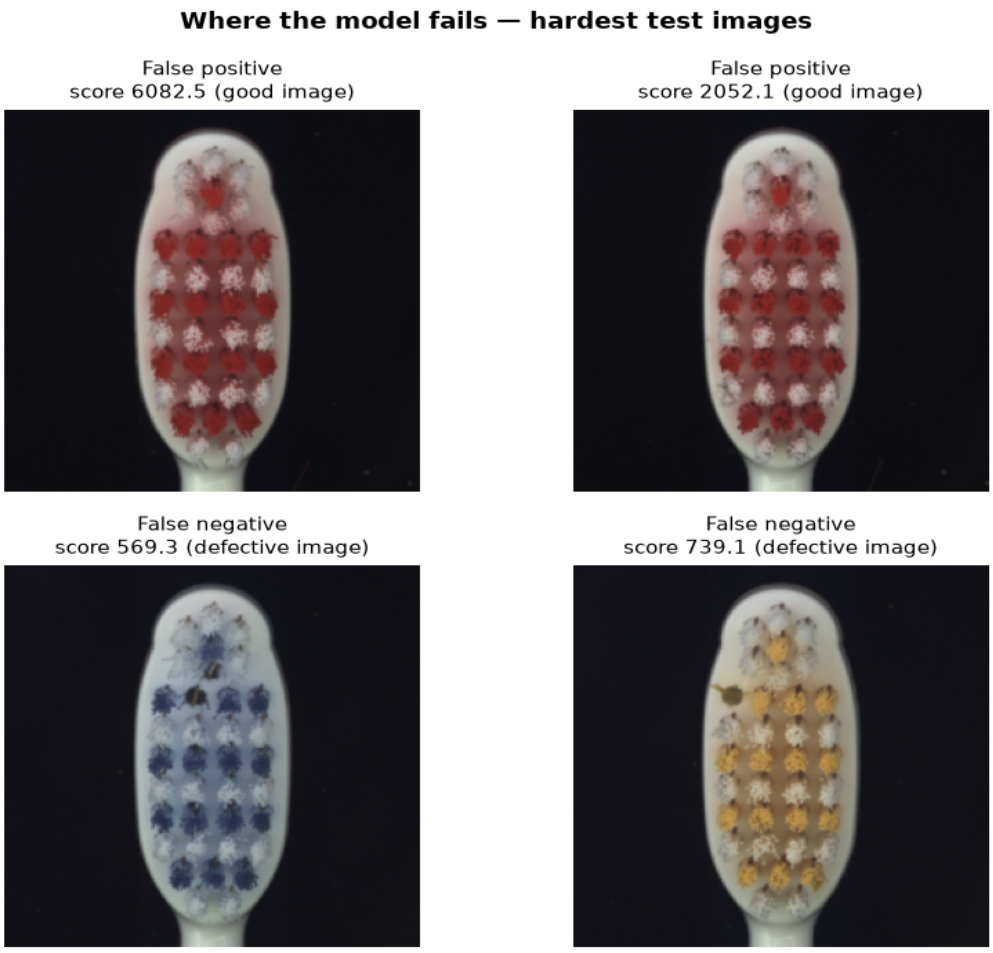


Figure 9 — Where the model fails: the two good images with the highest anomaly scores (false positives) and the two defective images with the lowest scores (false negatives).

Why does this project still belong in my portfolio? Because an independent, from-scratch re-implementation of a CVPR paper, evaluated under the exact protocol as my own baseline, with the negative result analyzed instead of buried, demonstrates three things supervisors actually look for: method literacy (I can read a paper and build it), experimental discipline (same data, same metrics, no moving goalposts), and honesty about limits. Those are rarer — and more convincing — than another "we beat SOTA" claim.

7 What I would do next

If I continue this line of work, the priorities are clear, roughly in order:

1. Full fine-tuning on a GPU, following the paper exactly — unfreezing the early layers is the single change most likely to close the image-level gap on texture-like defects.

2. A learned forge. Replace the hand-made cut-paste/scar families with a small diffusion model conditioned on normal images. Realistic synthetic defects would shrink the synthetic-to-real gap that Section 6 identifies as the core weakness.

3. More categories. Test on MVTec textures (carpet, leather, wood) as well as objects. Textures behave very differently from objects, and a method should prove itself on both.

4. Harder pretext tasks. Stronger augmentations, more forge families, or a contrastive objective, so the representations keep improving instead of saturating at epoch 7.

5. Few-shot extension. If even 5–10 real defective images are available, use them to calibrate the scorer — a practical middle ground between the zero-defect regime of this project and fully supervised learning.

References

[1] C. Li, S. Sohn, J. Yoon, T. Pfister, "CutPaste: Self-Supervised Learning for Anomaly Detection and Localization", *Proc. CVPR*, 2021.

[2] T. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, P. Gehler, "Towards Total Recall in Industrial Anomaly Detection", *Proc. CVPR*, 2022.

[3] P. Bergmann, M. Fauser, D. Sattlegger, C. Steger, "MVTec AD — A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection", *Proc. CVPR*, 2019.

[4] K. Song, Y. Yan, "A noise robust method based on completed local binary patterns for hot-rolled steel strip surface defects", *Applied Surface Science*, vol. 285, 2013 (dataset used in Project 2, Track A).